

ANWENDUNGSSYSTEME PROGI RECAP

Anne

CONCURRENCY

Mehrere Teile eines Programms können **überlappend** oder **gleichzeitig** ausgeführt werden.

Wichtig:

- Concurrency heißt **nicht zwingend**, dass alles exakt gleichzeitig läuft
- auch **schnelles Abwechseln** (Pseudo-Parallelität) zählt dazu

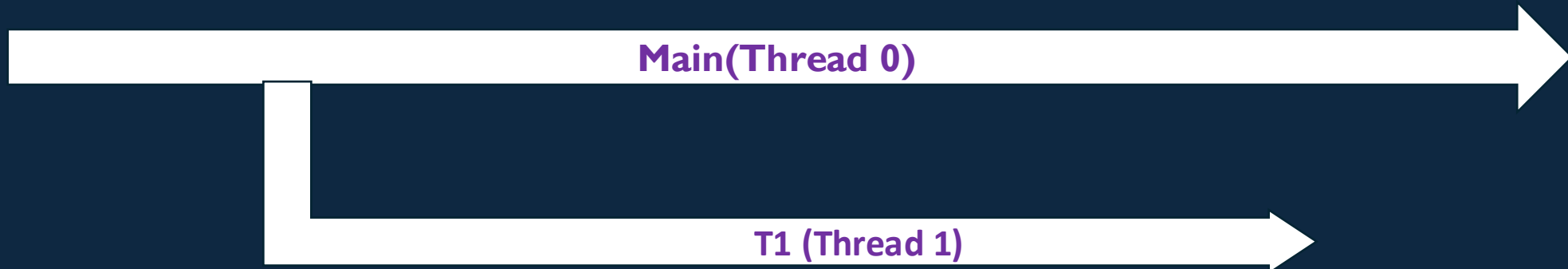
BIS JETZT

- Thread 0



JETZT

- Thread 0 erstellt thread 1

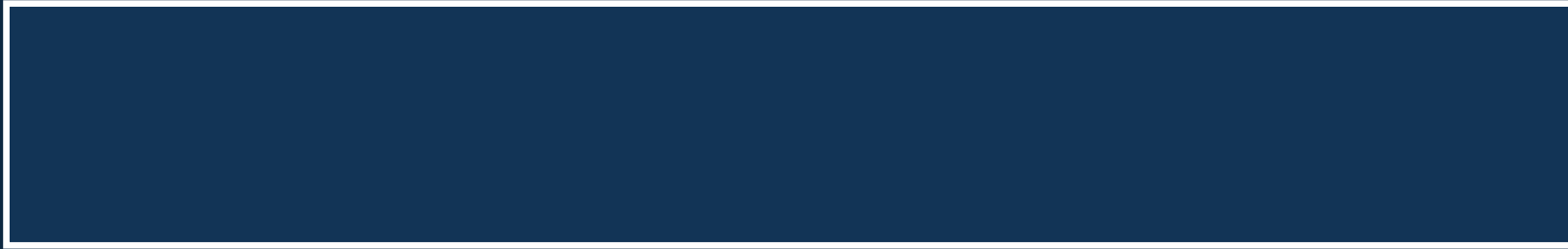


```
public static void main(String[] args) {
```

```
    Thread t1 = new SimpleThread();
```

```
        t1.start();
```

```
}
```



```
class SimpleThread extends Thread{  
  
    @Override  
    public void run(){  
        System.out.println(" I am a simple thread");  
    }  
}
```

```
public static void main(String[] args) {  
  
    Thread t1 = new SimpleThread();  
  
        t1.start();  
}
```

MÖGLICHKEITEN FÜR UMSETZUNG

- Thread (Klasse)
- Thread t1 = new MyThread();
→ Falls MyThread von Thread erbt
- Runnable (Interface)
- Thread t2 = new Thread(new MyRunnable());
- → Falls Runnable implementiert
- Hier können wir auch von anderen Klassen erben

Threads startet man IMMER mit **.start()**

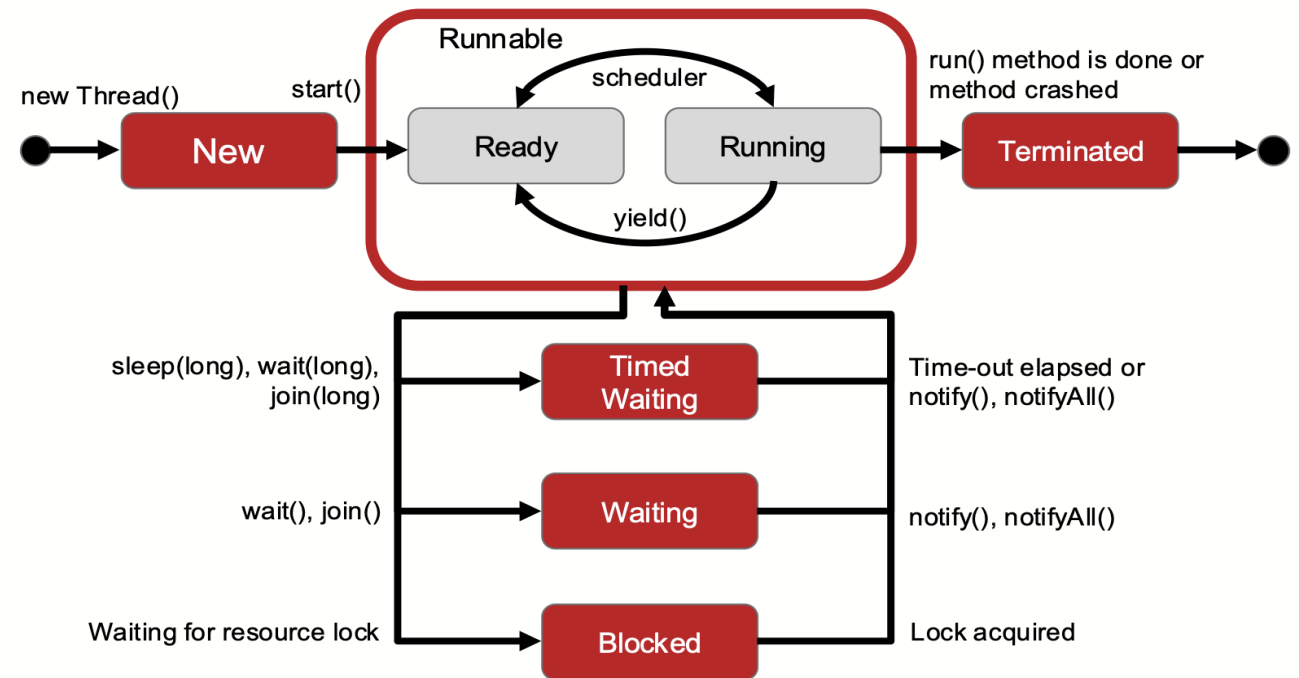
NIEMALS mit **.run()**

→ Methode läuft, aber nicht parallel

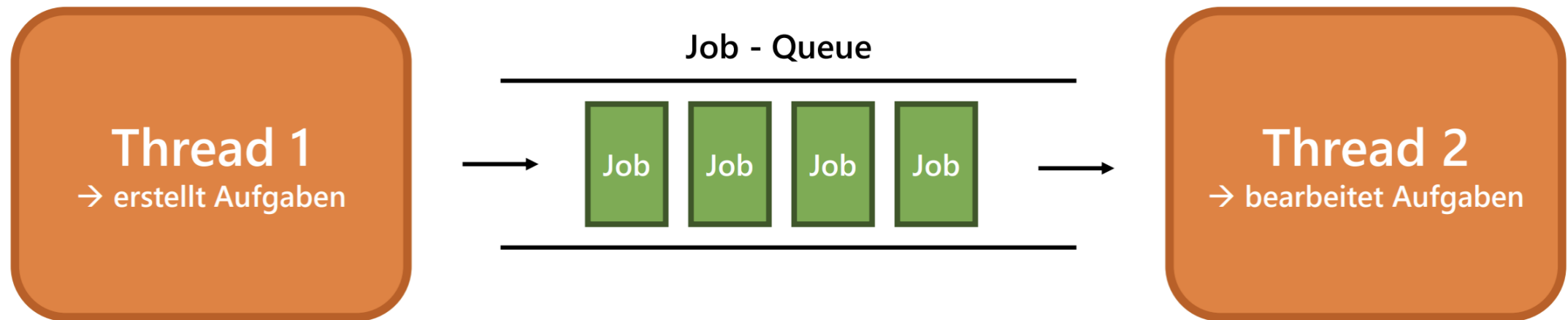
STATES EINES THREADS

Der **Scheduler** des Betriebssystems entscheidet, welcher Thread wann läuft.

Methode	Bedeutung
<code>start()</code>	Thread starten
<code>run()</code>	Code des Threads
<code>sleep()</code>	aktueller Thread wartet
<code>join()</code>	auf anderen Thread warten
<code>interrupt()</code>	Abbruchsignal senden



PRODUCER CONSUMER PROBLEM



Was ist das kritische Objekt?

2 Probleme

1. Was wenn Queue voll?
2. Was wenn Queue leer?

SYNCHRONIZED()

Wie kann man synchronized benutzen?

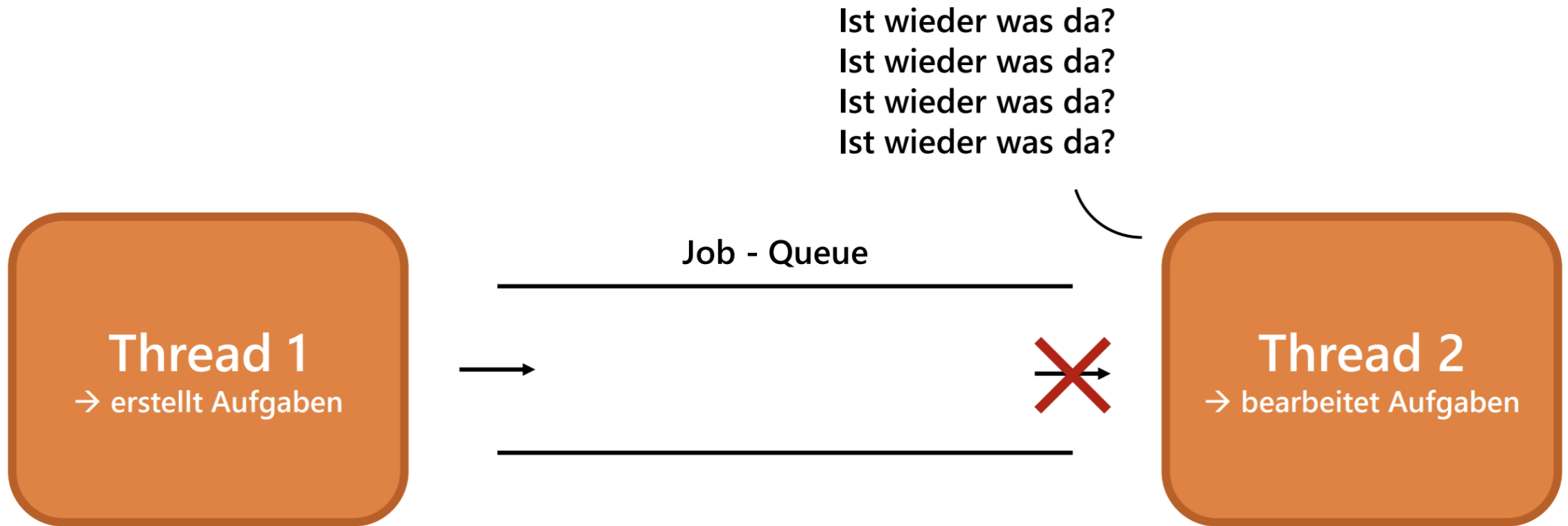
1. synchronized eine Ganze Methoden

```
synchronized void getRessource(){  
  
    //do something  
  
}
```

2. synchronized-Block

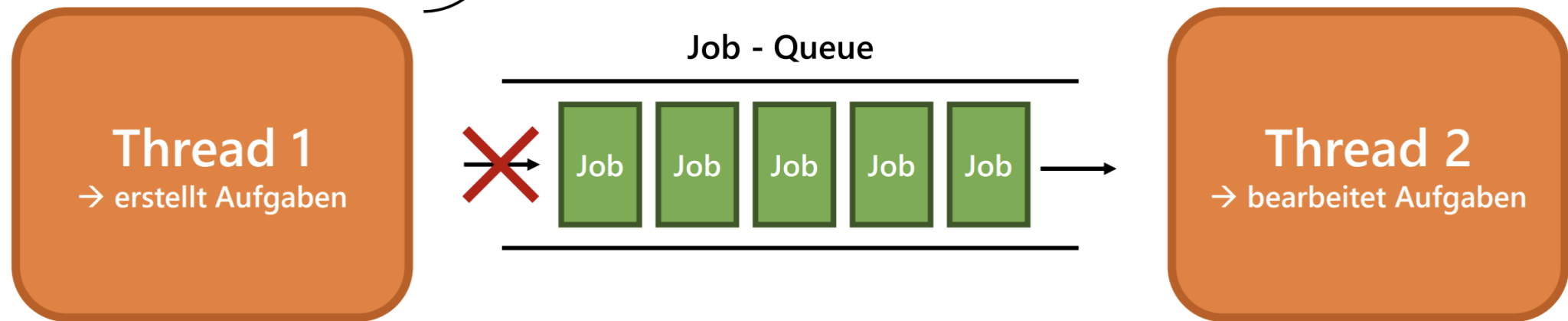
```
synchronized (lock) {  
  
    //do something  
  
}
```

Queue leer



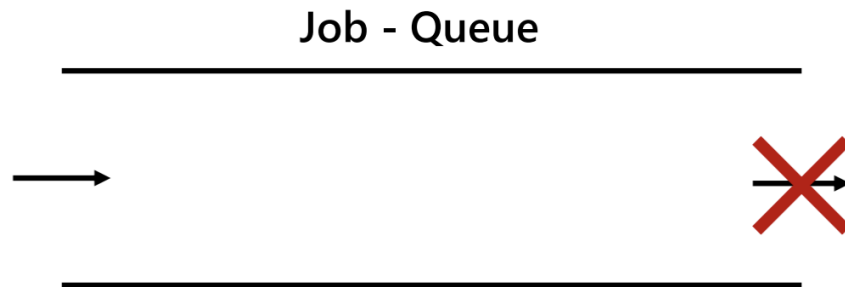
Queue voll

Ist schon wieder Platz?
Ist schon wieder Platz?
Ist schon wieder Platz?
Ist schon wieder Platz?



LÖSUNG

36

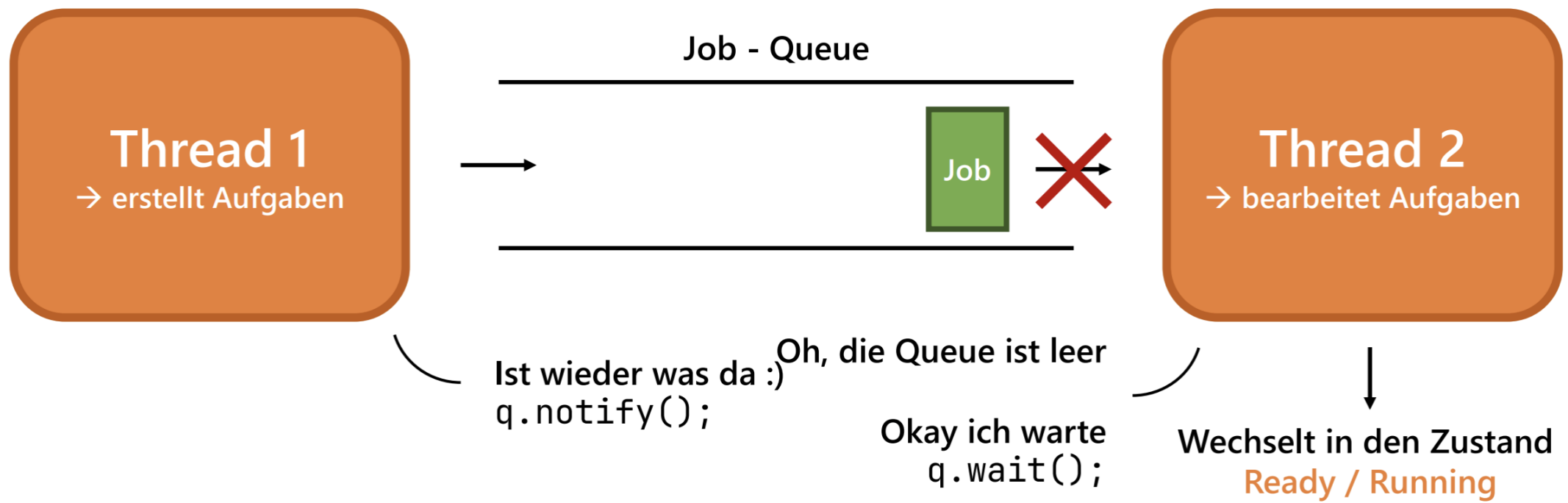


Oh, die Queue ist leer

Okay ich warte
`q.wait();`

LÖSUNG

36



SOCKET/STREAMS

- Streams: Datenfluss
- Sockets: Verbindung -> nutzen Streams

WIE MACHEN WIR DAS IM CODE?

```
BufferedReader br = new BufferedReader(new FileReader("input.txt"))
String result = "";
String line = "";
while ((line = br.readLine()) != null) {
    result += line + "\n";
}
System.out.println(result);
```

→ Sobald null eingelesen wird, weiß ich, dass der String vollständig ist

SOCKETS / STREAMS

BufferedReader macht aus Zeichen **Zeilen**.

CLIENT SEITE: SOCKET

- `Socket s = new Socket("localhost", 9999);`
- Socket erstellen = Verbindung versuchen
- -> der konstruktor versucht sofort zu verbinden
- -> wenn es nicht klappt-> Exception

SERVER SEITE:

- **Socket:**
- Socket ist Verbindung, aber der Client initiiert
- **ServerSocket** = Rezeption / Front Desk

Server

Client

```
ServerSocket server = new ServerSocket(9999);  
Socket client = server.accept();
```

```
Socket s = new Socket("localhost", 9999);
```

accept() blockiert: es wartet, bis ein Client sich verbindet.

STREAMS

- Für uns alles über Streams-> als Client brauchst du Streams vom und zum Server
- -> aus dem Socket bekommst du
- **InputStream in = mySocket.getInputStream();**
- **OutputStream out = mySocket.getOutputStream();**
- Wenn Fertig: **mySocket.close();**
- -> schließt verbindung und beide Streams

WELCHER STREAM IST WELCHER?

InputStream: Daten von der anderen Seite zu mir



OutputStream: Daten von mir zur anderen Seite

AUFGABE

Aufgabe 13: *Daytime Protocol*

Mithilfe des Daytime Protocols² kann standardisiert die aktuelle Uhrzeit abgefragt werden. Implementieren Sie ein Programm, dass von `time.nist.gov` auf Port 13 die Uhrzeit abfragt. Hinweis: Das Protokoll ist sehr einfach. Sie brauchen nichts zu senden, sondern einfach nur den Output auf der Konsole ausgeben.

GITHUB – CODE KOMMT SPÄTER

